简历工坊

从「投什么」到「怎么投」的求职 Agent

传智杯 · AI 应用开发赛道 · 技术文档

作品名称	简历工坊 Resume Workshop
参赛赛道	AI Agent 智能助手
技术栈	Next.js 16 / React 19 / TypeScript
文档版本	v1.0 · 2026-10-30

目录

1　项目概述

2　需求分析

3　系统架构设计

4　AI 技术方案

5　核心功能实现

6　测试与部署

7　总结与展望

附录 A　关键代码索引

附录 B　环境变量与命令

1 项目概述

1.1 背景

求职是一个高度重复、又高度个体化的过程：同一份简历要投几十个岗位，每个岗位的要求都不一样；求职者既要判断「这个岗位值不值得投」，又要在投之前把简历调整到位，还要记录投了哪些、什么时候跟进。市面上的产品通常只解决其中一段——简历工具只做排版，招聘平台只做岗位分发，AI 写作工具只做润色，没有工具把「看懂岗位要求 → 判断匹配 → 定向修改 → 生成材料 → 记录投递」串成一条可解释的链路。

更现实的问题是：求职者往往不知道自己差在哪。「投了 50 份没有回音」的背后，可能是方向不对，也可能是简历里根本没有 JD 要的那几个词。这类判断需要把 JD 和简历放在一起看，而这正是大模型擅长、传统工具做不了的事。

1.2 问题定义

我们把求职者在投递前的动作拆成三个断点，产品对应解决三点：

1. 看不懂岗位：JD 写得很虚，「熟悉工程化」「有性能优化经验」到底指什么，求职者无法自行判断。
2. 改不动简历：知道要突出某段经历，但不知道怎么写才贴合这个岗位，于是选择不改。
3. 记不住投递：投过谁、投的哪版简历、什么时候该跟进，全靠脑子和备忘录。

1.3 产品定位与目标用户

简历工坊是一个「本地优先」的求职工作台：简历与投递记录全部保存在浏览器本地，服务端只承担 AI 代理的角色，不落库、不收集联系方式。目标用户是应届生与 1–5 年经验的转岗者——他们有明确的岗位方向，但没有成熟的求职方法论，也缺少简历优化的经验。

产品形态是一个 Web 应用，另附带一个 Chrome 扩展：扩展负责把真实招聘网站上的 JD 抓进来，并在投递表单页完成基础信息填充，两者通过本地消息桥连接，不经过任何中间服务器。

1.4 与同类产品的差异

维度	常见简历工具	招聘平台	AI 写作工具	简历工坊
岗位数据	无	平台自有	无	浏览器扩展抓真实 JD
匹配判断	无	平台黑盒推荐	无	可解释打分 + 缺口
材料生成	模板排版	无	单点润色	Agent 四步生成
投递记录	无	平台内	无	本地清单 + 跟进

数据归属	云端账号	平台	云端	浏览器本地

2 需求分析

2.1 用户画像与典型场景

用户	特征	核心诉求	对应能力
应届生	项目少、无实习或只有一段	不知道简历够不够格	匹配评分 + 缺口提示 + 起草
转岗者	有经验但方向不对口	如何把旧经历讲成新岗位要的样子	按 JD 定向改写 + 事实核对
海投者	一次投几十个岗位	批量管理、不漏跟进	待投清单 + 批量导入 + 跟进日期
谨慎型	不愿上传简历到云端	数据不出本机	本地存储 + AI 只发必要字段

2.2 功能需求

模块	能力	说明
简历编辑	8 套模板、字体搭配、中英双语	即时预览、撤销重做、多份简历隔离
匹配引擎	26 个岗位画像、核心要求分组	给出分值构成与可点击的缺口
待投清单	批量导入 JD、按匹配度排序	准备弹窗内完成材料与状态记录
求职 Agent	计划 → 执行 → 轨迹	两步本地、两步 AI，采用前需人工确认
浏览器扩展	抓取 JD、填充表单	不自动提交、跳过身份证与密码
导出交付	PDF 打印、ATS 纯文本	分页控制、关键词覆盖检查

2.3 非功能需求

- 隐私：简历默认不出本机；调用 AI 时只发送当前板块文本、岗位关键词与参考 JD，不发送姓名、电话、邮箱与整份简历。
- 可用性：核心编辑与匹配功能不依赖网络与 API Key；未配置 Key 时给出配置指引而不是报错。
- 性能：AI 输出采用流式渲染，首字延迟可控；页面为静态渲染 + 局部水合，离线可打开。

- 可维护性：匹配规则、岗位画像、提示词与 UI 解耦，改规则不需要改组件；关键逻辑有单元测试与浏览器端回归。

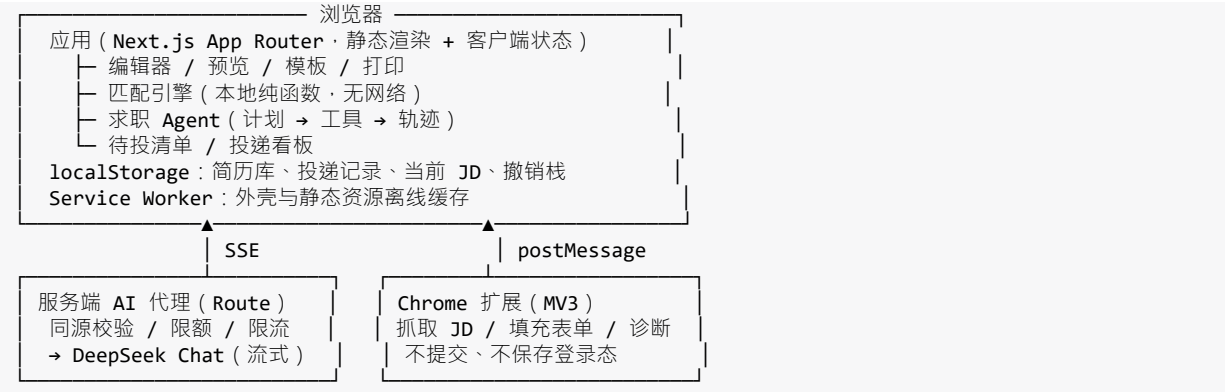
2.4 边界与非目标

产品不做自动投递。招聘平台没有公开的职位 API，抓列表页与自动提交都违反平台条款，也会给用户带来封号风险；因此扩展只处理「当前打开的那个岗位页」，提交动作始终由用户完成。此外不做账号体系、云同步与机器翻译——英文内容由用户填写，缺失时回退中文。

3 系统架构设计

3.1 总体架构

系统分为四层，边界清晰：浏览器端应用层负责编辑、匹配与状态管理；服务端 AI 代理层只做「鉴权 → 限额 → 校验 → 转发 → 流式回传」；本地数据层用 localStorage 承担持久化；浏览器扩展作为独立进程，通过页面消息桥与应用通信。



3.2 关键数据流

匹配与投递（全程本地）：

```
JD 文本 → analyzeJd（词库覆盖率、年限/学历）→ 与岗位画像交叉 → 分值构成 + 缺口
        → 待投清单按分排序 → 准备弹窗（补技能 / ATS / Agent / 求职信）→ 标记已投
```

AI 生成（流式）：

```
用户点「执行」→ 组装请求（板块 / 关键词 / 参考 JD / 语言）
  → POST /api/ai/generate → 同源校验 → 64KB 限额读取 → 参数校验 → 限流
  → DeepSeek（stream=true）→ SSE 逐块回传 → 前端渲染
  → 完成后本地事实核对 → 人工「采用」→ 写入简历（进撤销栈）
```

3.3 数据模型与本地存储

键	内容	约束
resume-workshop:v1	当前简历（含翻译覆盖层）	schemaVersion 校验，非法值回退
resume-workshop:library:v1	多份简历与当前选中项	单份 1.5MB 预算
resume-workshop:applications:v1	投递记录	上限 300 条，写入前先读最新
resume-workshop:jd	当前 JD（供 AI 参考）	截断 4000 字
sessionStorage:history	撤销 / 重做栈	20 步 / 1.5MB，关闭标签页清空

3.4 技术选型与理由

技术	用途	选择理由
Next.js 16（App Router）	页面与服务端路由	静态渲染 + 局部水合，AI 接口与页面同域，省去跨域与鉴权
React 19 + TypeScript	交互与类型安全	strict 模式，组件与规则共享类型；编译期拦截数据形状错误
Tailwind CSS 4	样式	与打印样式分离清晰，主题色以 CSS 变量注入
localStorage + SW	持久化与离线	无账号、无服务端存储，符合隐私目标；外壳可离线
Vitest + Testing Library	单元与组件测试	与 Vite 生态一致，jsdom 下可测弹窗与表单
Puppeteer	端到端回归	可驱动真实 Chrome，验证打印、PDF 页数与扩展脚本
Chrome MV3	扩展	官方支持的扩展形态，权限最小化（tabs + storage）

3.5 工程结构

```
src/
  app/              页面与 /api/ai/* 路由
  components/
    resume/         编辑器、预览、模板、导出对话框
    jobs/           匹配面板、待投清单、求职 Agent、求职信
    ui/             Dialog / Button / Field 等基础组件
  hooks/            useResume / useResumeDocuments / useApplications
  lib/
    matching.ts     匹配引擎（要求分组、分值、缺口）
    jobProfiles.ts  26 个岗位画像与核心要求分组
    agent/          求职 Agent：计划、工具、轨迹
    ai/             提示词、SSE 客户端、事实核对、服务端工具
```

```
    eval/          离线评测与报告渲染
scripts/           qa.mjs（端到端）、eval.mjs、pack-extension.mjs
extension/         Chrome MV3 扩展（抓取 / 填表 / 诊断）
```

4 AI 技术方案

本章回答三个问题：为什么选这个模型、怎么把它接进产品、怎么保证输出可用。这也是本项目与「调一次接口就叫 AI 应用」的根本区别。

4.1 模型选型理由

候选	结论	理由
DeepSeek Chat	选用	中文表达自然、支持流式与 JSON 输出、价格低，适合高频短文本改写；国内访问稳定
GPT-4 级通用模型	未选	成本高出一个量级，用于「改一段简历」性价比低；跨境访问不稳定
本地小模型	未选	中文长文本改写质量不足，且要求用户装环境，与 Web 形态冲突
模板 + 规则生成	作为兜底	无 Key 时仍可用，但无法针对 JD 调整措辞，只能保证流程不中断

选型的核心判断是「任务形态」：本产品的 AI 调用是大量短文本、强约束、需要中文语感的任务，而不是长文档推理。这类任务对模型规模不敏感，对提示词结构与输出约束更敏感，因此把预算放在「约束设计」而不是「更大的模型」上。

4.2 集成方式

三类调用共用一个流式通道，按板块区分：

接口	用途	输出约束
/api/ai/generate	单段写作（简介 / 经历 / 项目 / 求职信）	纯文本，按板块限制长度；双语用 ===EN=== 分隔
/api/ai/full	整份简历生成	JSON（response_format=json_object），缺字段留空
/api/ai/status	是否已配置 Key	只返回布尔与模型名，不泄露 Key

服务端处理链路（所有 AI 接口一致）：

1. 同源校验：检查 Sec-Fetch-Site / Origin 与 Host，跨站调用直接 403。

2. 限额读取：边读边计数，超过 64KB 立即中断，避免大包拖垮进程。

3. 参数校验：板块、语气、语言、字段长度逐项校验，非法值返回 400。

4. 限流：按可信客户端 IP 计数（取 X-Forwarded-For 最右一跳），桶数量上限一万并回收。

5. 转发与回传：以 SSE 逐块回传，错误统一映射为可读文案，前端按 code 展示。

4.3 提示词与上下文设计

- 角色约束：明确「你是资深 HR / 简历顾问」，限定只改写给定板块，不新增经历。
- 素材边界：提示词声明「只能用用户提供的事实」，数字、公司、技能不得编造；参考 JD 只用于调整重点，不作为事实来源。
- 结构化输出：整份生成要求 JSON 且字段固定，解析失败时回退到原文并提示，不写入半成品。
- 语言策略：中文 / 英文 / 双语三种输出；双语用独占一行的 ===EN=== 分隔，前端据此拆回两个字段。
- 长度控制：按板块设定字数上限并在提示词中重申，服务端再截断一次，双保险。

4.4 质量保障：事实核对

模型最危险的行为不是写得差，而是「写得很像但事实不对」——把 45% 写成 65%，或者凭空加一个没做过的技术。为此每次生成后都做一次本地核对：

1. 数字核对：抽取生成文本中的数字与单位，逐个在素材中查找，找不到即提示。

2. 技能核对：用技能词库 + 同义词匹配生成文本，命中素材之外的技能即提示。

3. 名称核对（整份生成）：学校、公司、项目名必须出现在用户自述中。

核对结果以面板形式列出，不阻断采用——同义改写与跨语言表达可能误报，由用户判断；但「采用」按钮旁会显示待核对数量，避免一键跳过。

4.5 隐私与安全边界

- 不发送姓名、电话、邮箱与整份简历；只发送当前板块文本、岗位 / 年限 / 关键词与参考 JD。
- API Key 只存在于服务端环境变量，前端无法读取；`.env*` 被 .gitignore 排除，仅提交模板。
- 输出以纯文本渲染，不使用 innerHTML；扩展传入的内容同样按文本处理。
- 限流与限额同时限制单请求体积与请求频率，降低被刷的风险。

4.6 求职 Agent：计划、工具与轨迹

Agent 是本项目把 AI 从「写作助手」提升为「流程主导者」的关键设计。它不是一个聊天框，而是一条可观察、可中断、可回滚的执行链路。

四步计划：

步骤	类型	做什么	输出
解析岗位要求	本地	词库识别 JD 关键词、年限与学历	匹配分 + 已覆盖 / 缺失
诊断能力差距	本地	对照岗位画像的核心要求分组	缺口 + 分值构成
按 JD 改写简介	AI 流式	只用已有事实，面向该岗位重写	预览 + 事实核对 + 采用
生成求职信	AI 流式	同一份材料写给该岗位	预览 + 复制

三个设计取舍：

1. 计划不依赖模型：步骤由简历与 JD 现场计算（简介为空是「起草」，有内容是「改写」），没有 Key 也能跑完前两步，演示不会断链。
2. 工具边界清晰：本地工具不出网，AI 步骤走既有流式接口；每步都是纯函数或单向请求，便于测试与替换。
3. 人工在环：AI 结果必须经过事实核对、点击「采用」才写入简历，且写入进入撤销栈；不采用则丢弃。

执行过程记录每一步的状态、耗时与工具名，形成「工具调用轨迹」。轨迹既是调试信息，也是向用户解释「AI 刚才做了什么」的依据。

4.7 性能与成本优化

- 流式渲染：先出首字再补全，长文本感知延迟显著下降；服务端关闭代理缓冲。
- 请求裁剪：只发必要字段，关键词取匹配结果的前 10 个，避免无意义的长上下文。
- 减少无效生成：匹配分组把「同族技能」合并，改写时不会去补用户根本不需要的 Vue。
- 降级路径：未配置 Key 时禁用 AI 步骤并给出配置指引；AI 失败不阻塞本地匹配与投递记录。
- 缓存与静态化：页面静态渲染，AI 接口不进入离线缓存，避免把敏感响应写进缓存。



图 1　求职 Agent：四步计划、事实核对与工具调用轨迹

5 核心功能实现

5.1 简历编辑与多文档

编辑器采用「受控状态 + 本地持久化」：所有修改先落到内存，再由保存副作用写入 localStorage。为避免多标签页互相覆盖，未编辑过的标签页会跟随最新版本，编辑过的才暂停保存并提示冲突；撤销栈按操作合并，刷新后从 sessionStorage 恢复，关闭标签页清空。

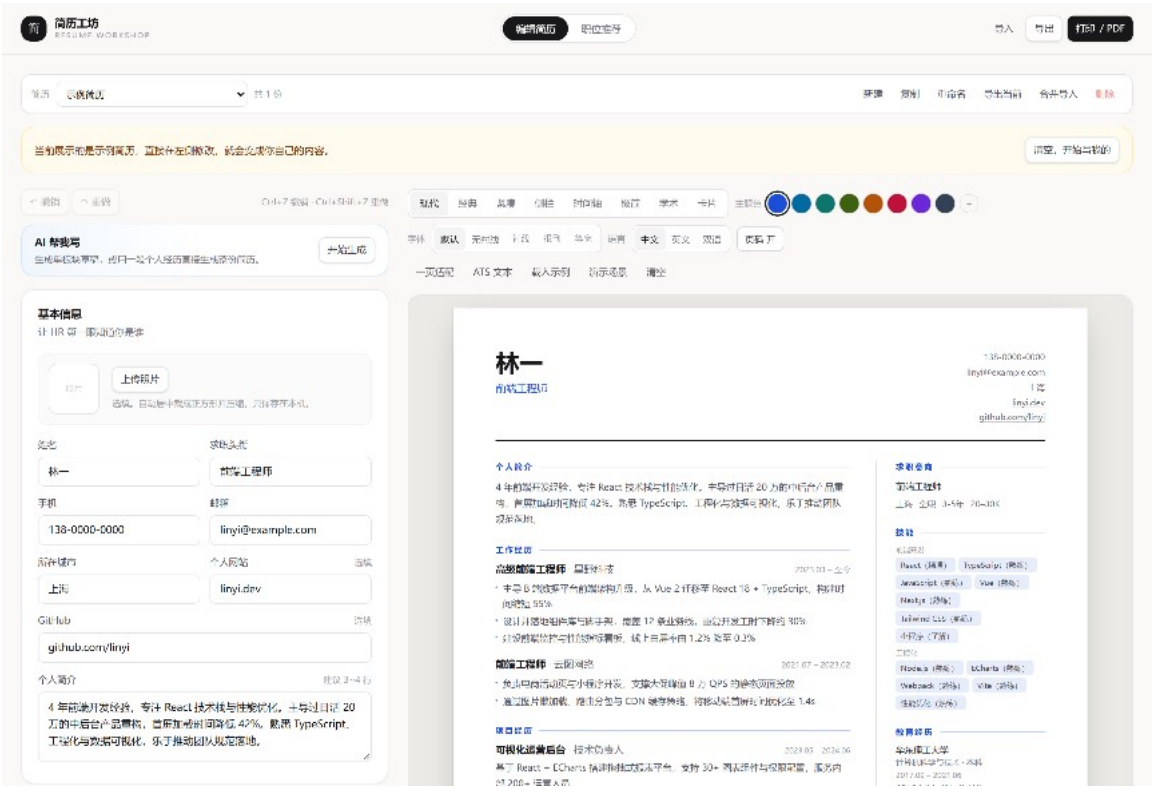


图 2　编辑页：左侧表单、右侧实时预览，可直接打印为 PDF

5.2 匹配引擎

匹配分由三部分构成，相加即总分：核心要求（55）+ 加分技能（20）+ 简历完整度（25）。核心要求不是「一个技能一项」，而是按「同族技能任选其一」分组——前端只会 React 或 Vue 之一，不应被判为缺失一半。分组同时让缺口提示变得可操作：缺失项是「浏览器与性能优化」这种要求，而不是硬塞给用户一个「Vue」标签。

```
核心要求分组示例（frontend）：
  [JavaScript, TypeScript]  [React, Vue]  [HTML, CSS]
  [工程化, Webpack, Vite, Rollup, 组件库, CI/CD]
  [浏览器, 兼容, 渲染, 首屏, 性能优化]
→ 组内任一命中即满足该项要求；缺失时报告组内真实技能，抽象要求只展示不可点击
```

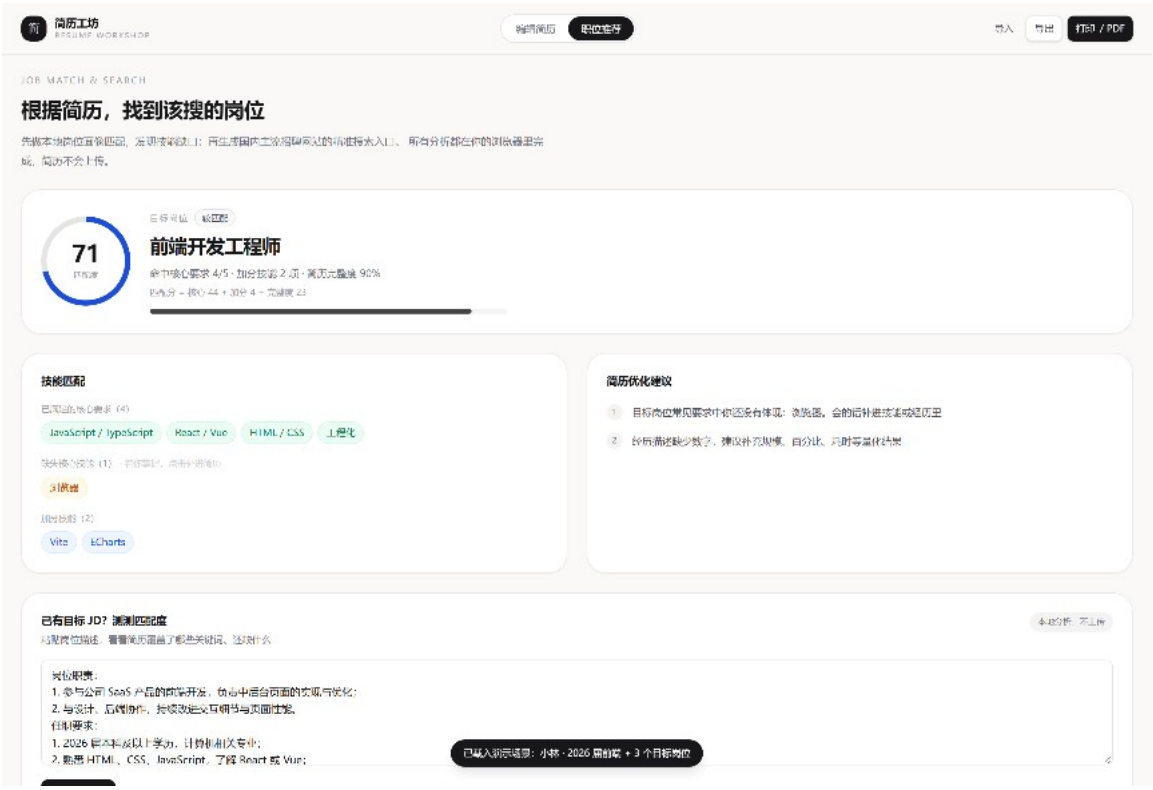


图 3　职位推荐：匹配分、核心要求命中、可点击的缺口与优化建议

5.3 待投清单

支持一次粘贴多条 JD（以 --- 分隔，首行可写「公司 | 岗位」），解析时做上限与截断保护。清单按当前简历的匹配度排序，点「准备」可在同一弹窗内完成：补缺口、下载 ATS 文本、按 JD 打开 AI 写作、启动求职 Agent、生成求职信、标记已投并设置跟进日期。

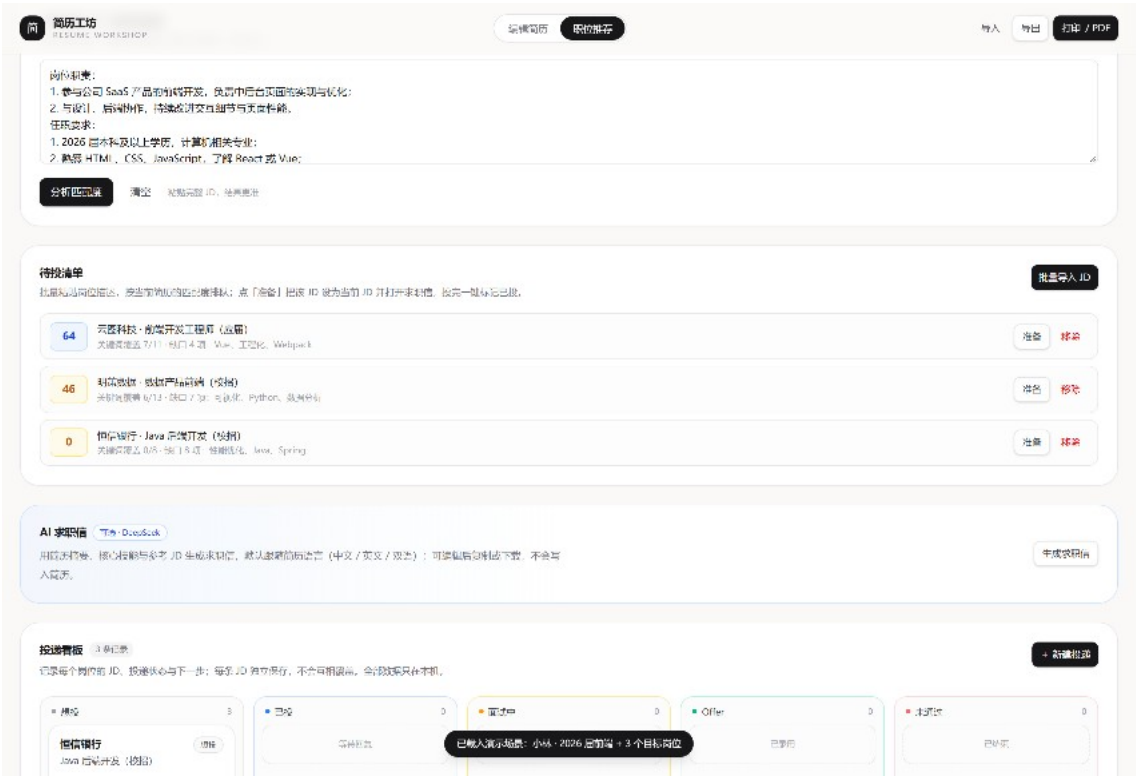


图 4　待投清单：批量导入的岗位按匹配度排序，可直接进入准备流程

5.4 浏览器扩展

扩展要解决的是「职位数据从哪来」。抓取采用四层兜底，越靠前越可靠：

1. JSON-LD：页面的 schema.org JobPosting 结构化数据（猎聘等站点提供）。
2. 站点选择器：按平台维护候选选择器，命中即用。
3. 标题解析：从「岗位招聘_公司招聘-站点」中取公司名，避免抓到相似职位卡片。
4. 正文兜底：按「职位描述」等标题文字向上找容器，适配没有稳定类名的页面。

同时提供「诊断选择器」：只读页面，回报每个字段命中的来源与每条选择器的命中数量，便于在真实站点上定位失效规则。填表只写姓名、联系方式等文本字段，跳过身份证、银行卡、密码与验证码，且不提交任何表单。

5.5 导出与打印

打印使用 CSS @page 控制纸张与页边距，页码由浏览器生成。双栏模板在打印时改为「浮动侧栏 + 固定栏宽正文」，解决 CSS Grid 无法跨页导致整块内容被推到第二页的问题；侧栏超过一页时，其底色随浮动盒延续。ATS 导出为线性纯文本 / Markdown，并按 JD 给出关键词覆盖提示。

6 测试与部署

6.1 测试分层

层次	规模	覆盖内容
单元测试（Vitest）	265 个用例 / 27 个文件	存储归一化、匹配与分组、JD 解析、提示词、事实核对、Agent 计划与工具、评测
组件测试（jsdom）	并入上项	弹窗焦点与可访问性、板块布局、表单交互
端到端（Puppeteer）	237 项检查	打印与 PDF 页数、多标签页、待投清单、Agent、扩展脚本、离线缓存
真实站点验证	3 个平台	智联 / BOSS / 猎聘用真实岗位页验证抓取逻辑

6.2 离线评测结果

为避免「AI 效果无法验证」，项目内置了可复现的离线评测（npm run eval），结果写入 docs/eval-report.md，同时作为回归基线。

指标	口径	结果
岗位画像识别	26 个画像各自的技能构造简历，能否认回同一画像	26 / 26
JD 覆盖率（上界）	同画像 JD 与技能齐全的简历比对	85 / 100
核心要求分组收益	同族只留一项时，逐项口径 vs 分组口径	82% → 100%
手写 JD 画像识别	5 段真实招聘文案	5 / 5
手写 JD 缺口提示	是否给出具体技能缺口	5 / 5
事实核对检出	20 条注入的素材外数字 / 技能	20 / 20
事实核对误报	5 条干净文本	0 / 5

其中「分组收益」最能说明问题：同一位只会 React 的求职者，逐项口径给出 82%，分组口径给出 100%——这 18 个百分点正是许多简历工具「显得不匹配」的原因。

6.3 CI

GitHub Actions 上执行：lint → 类型检查 → 单元 / 组件测试 → 生产构建 → 启动 next start → 完整浏览器 QA，失败上传截图。端到端检查全部针对生产构建，避免开发模式下的行为差异。

6.4 部署

推荐 Vercel：仅需配置 DEEPSEEK_API_KEY，其余零配置。自有服务器用 Node + nginx，注意三点：保留 Host（同源校验依赖它）、关闭代理缓冲（SSE 流式）、多层代理时用 TRUSTED_PROXY_HOPS 指定可信跳数。限流是单进程内存实现，多实例部署需替换为共享存储。

7 总结与展望

7.1 技术创新点

1. 核心要求分组 + 可解释打分：把「岗位要求」建模为同族技能组，并给出可核对的分值构成与缺口，而不是黑盒百分比。
2. AI 结果的事实核对与人工在环：生成后本地比对数字 / 技能 / 名称，必须人工采用才写入，且写入可撤销。
3. 不依赖模型的 Agent 计划：计划由简历与 JD 现场计算，保证无 Key 也能演示与使用，AI 只承担它擅长的两步。
4. 真实站点的四层抓取兜底 + 诊断模式：JSON-LD → 选择器 → 标题 → 正文兜底，并可在真实页面上回报命中来源。

7.2 应用创新点

1. 把「投什么、怎么投、投了没」串成一条链路，而不是单点工具；投递记录与材料在同一上下文中完成。
2. 本地优先的数据策略：简历与投递记录不出浏览器，AI 只接触必要字段，在隐私与能力之间取得平衡。
3. 合规的自动化边界：扩展只读当前岗位页、只填文本字段、不提交，规避平台条款与封号风险。

7.3 局限

- 评测样本规模小且部分由画像生成，存在循环性，结论用于回归而非绝对性能宣称。
- 事实核对只做字面比对，同义改写、跨语言表达可能漏报，也不判断内容是否真实。
- 扩展的站点选择器依赖页面结构，平台改版后需要按诊断结果修正；未登录场景下部分站点无法验证。
- 限流为单进程内存实现，多实例部署时需替换。

7.4 展望

下一步计划包括：把岗位画像扩到更多细分方向（AI 应用、直播电商、游戏）；让 JD 匹配也采用要求组口径；为 Agent 增加「多轮追问」——当简历信息不足时主动询问用户，而不是直接生成；以及在合规前提下支持更多招聘平台的表单填充。

附录 A 关键代码索引

文件	职责
src/lib/matching.ts	匹配引擎：分组计分、分值构成、缺口与建议
src/lib/jobProfiles.ts	26 个岗位画像与核心要求分组
src/lib/agent/plan.ts	Agent 四步计划（随简历与 JD 变化）
src/lib/agent/tools.ts	本地工具：解析 JD、诊断差距
src/lib/ai/prompt.ts	单段 / 整份提示词与语言分隔
src/lib/ai/factCheck.ts	生成结果的事实核对
src/lib/ai/http.ts	同源校验、限额读取、限流、SSE 编码
src/lib/eval/report.ts	离线评测与报告渲染
extension/content-job.js	抓取四层兜底、表单填充、诊断
scripts/qa.mjs	端到端回归（打印 / 多标签页 / Agent / 扩展 / 离线）

附录 B 环境变量与命令

项	说明
DEEPSEEK_API_KEY	服务端密钥，必填；不得出现在前端或仓库中
DEEPSEEK_MODEL	模型名，默认 deepseek-chat
DEEPSEEK_BASE_URL	接口地址，默认官方地址
TRUSTED_PROXY_HOPS	可信代理跳数，默认 1
npm run dev / build / start	开发、构建、生产启动
npm test / qa / eval	单测、端到端回归、离线评测
npm run pack:extension	打包 Chrome 扩展（校验 manifest 引用与图标尺寸）